# 合肥工業大學

2025~2026 学年 第一学期

# 《计算机体系结构》课程报告

## 缓存优化与安全研究

| | |
|---|---|
| 学　号 | 2023212351 |
| 姓　名 | 邓杰湧 |
| 专　业 | 物联网工程 |
| 班　级 | 23-1 班 |

2025 年 11 月

# 目录

计算机体系结构中的缓存优化与安全研究进展.....4

1.概述.....4

2.智能缓存管理：CHROME 框架的 holistic 优化.....4

2.1 设计背景：传统缓存管理的两大核心痛点.....4

2.1.1 策略孤立：三种核心技术 “各自为战”.....4

2.1.2 静态决策：无法适配动态工作负载.....4

2.2 核心技术架构：强化学习驱动的整体化设计.....5

2.2.1 缓存管理的强化学习建模.....5

2.2.2 多特征状态感知：精准捕捉内存访问行为.....6

2.2.3 并发感知的奖励机制：结合局部性与系统级反馈.....6

2.3 硬件实现细节：低开销与高实用性.....7

2.3.1 Q 表：高效存储状态 - 动作的 Q 值.....7

2.3.2 评估队列（EQ）：延迟奖励的 “记账本”.....8

2.3.3 采样训练：降低计算复杂度.....8

2.4 性能优势：低开销下的全面超越.....9

2.4.1 性能提升显著且稳定.....9

2.4.2 硬件开销极低.....9

2.4.3 扩展性优异.....10

3.缓存一致性协议：虚拟网络的死锁防护机制.....10

3.1. 研究背景：虚拟网络死锁问题与传统方案的缺陷.....10

3.1.1 传统方案的 “不充分性”：静态依赖链分析导致死锁.....10

3.1.2 传统方案的 “不必要性”：过度配置浪费资源.....11

3.2. 核心技术：形式化建模与死锁分析框架.....11

3.2.1 三大核心关系：刻画协议与网络的交互.....12

3.2.2 网络排队模型：queues 关系与死锁条件.....12

3.3. 协议分类与最小 VN 数量判定.....12

3.3.1 三类协议的 VN 需求差异.....12

3.3.2 CHI 协议的验证：2 个 VN 足够.....12

3.4. 实验验证：Murphi 模型与结果.....13

3.4.1 模型配置与验证方法.....13

3.4.2 关键实验结果.....14

4.安全高效缓存：Maya 的存储优化与攻击防护.....15

4.1 研究背景：安全缓存的存储与性能矛盾.....15

4.1.1. 传统随机化缓存的安全与性能局限.....15

4.1.2. Mirage 缓存的存储开销问题.....15

4.1.3. 关键观察：缓存数据的低重用特性.....15

4.2 核心设计：解耦架构与重用驱动的存储优化.....16

4.2.1. 标签与数据存储设计.....16

4.2.2. 插入与驱逐策略.....17

4.3 安全机制：抵御冲突攻击与共享内存攻击 ..... 17
4.3.1. 冲突攻击防御：避免集相联驱逐（SAE） ..... 18
4.3.2. 共享内存攻击防御：安全域隔离（SDID） ..... 18
4.3.3. 其他攻击防御补充 ..... 18
4.4 性能表现：多工作负载下的稳定性与优势 ..... 19
4.4.1. 同构工作负载性能 ..... 19
4.4.2. 异构工作负载性能 ..... 19
4.4.3. 扩展性与敏感性分析 ..... 19
4.5 硬件开销：存储、面积与功耗优化 ..... 19
4.5.2. 面积与功耗 ..... 20
4.5.3. 与安全分区技术的对比 ..... 20
总结 ..... 20
参考文献 ..... 20

# 计算机体系结构中的缓存优化与安全研究进展

## 1.概述

在现代计算机体系结构中，缓存作为衔接处理器与主存的关键部件，其性能与安全性直接影响整个系统的运行效率和稳定性。近年来，研究者们围绕缓存管理优化、死锁避免及安全防护等核心问题展开深入探索，三篇文档分别从智能缓存管理框架、虚拟网络死锁防护及安全高效缓存设计三个维度，呈现了该领域的前沿研究成果，为解决缓存系统面临的性能瓶颈与安全威胁提供了创新方案。

## 2.智能缓存管理：CHROME 框架的 holistic 优化

CHROME 作为面向多核心系统的创新缓存管理框架，核心是通过整体化设计与在线强化学习，解决传统缓存管理中策略孤立、适应性差的问题。以下从设计背景、核心技术架构、硬件实现细节、性能优势四个维度，对其进行详细拆解。

### 2.1 设计背景：传统缓存管理的两大核心痛点

在现代多核心处理器中，缓存作为衔接高速 CPU 与低速主存的关键部件，其管理效率直接决定系统性能。但传统方案存在两大显著缺陷，成为 CHROME 框架的核心设计动因：

#### 2.1.1 策略孤立：三种核心技术 “各自为战”

缓存管理的三大关键技术 ——缓存替换（决定淘汰哪些缓存块以容纳新数据）、缓存旁路（判断是否将新数据写入缓存，避免 “污染”）、数据预取（主动提前获取未来可能用到的数据），在传统研究中常被单独优化，忽略了三者的协同效应。例如：预取的数据块若被盲目写入缓存，可能挤占需求访问（处理器主动发起的访问）所需的缓存空间；而旁路策略若未考虑预取数据的准确性，会误将有用的预取块排除在缓存外。这种 “孤立设计” 导致缓存资源浪费，难以应对多程序并发运行时的复杂内存访问模式。

#### 2.1.2 静态决策：无法适配动态工作负载

传统方案（如经典的 LRU 算法、Hawkeye 的二进制重用预测）多基于人工经验设计静态策略，依赖 “应用访问模式稳定”“工作负载特征可预判” 等假设。但现代系统中，多核心同时运行的程序类型差异大（如 SPEC CPU 的计算密集型程序与 GAP 的图算法程序），且同一程序在不同运行阶段的内存访问行为也会动态变化。静态策略无法实时调整，导致在复杂场景下性能骤降。例如 Mockingjay 虽尝试整合替换与旁路，但因策略固定，在不同预取配置（如 L1 用 stride 预取、L2 用 streamer 预取）下，部分工作负载（如 soplex、wrf）的性能甚至低于传统方案。相关对比可以查看图 1

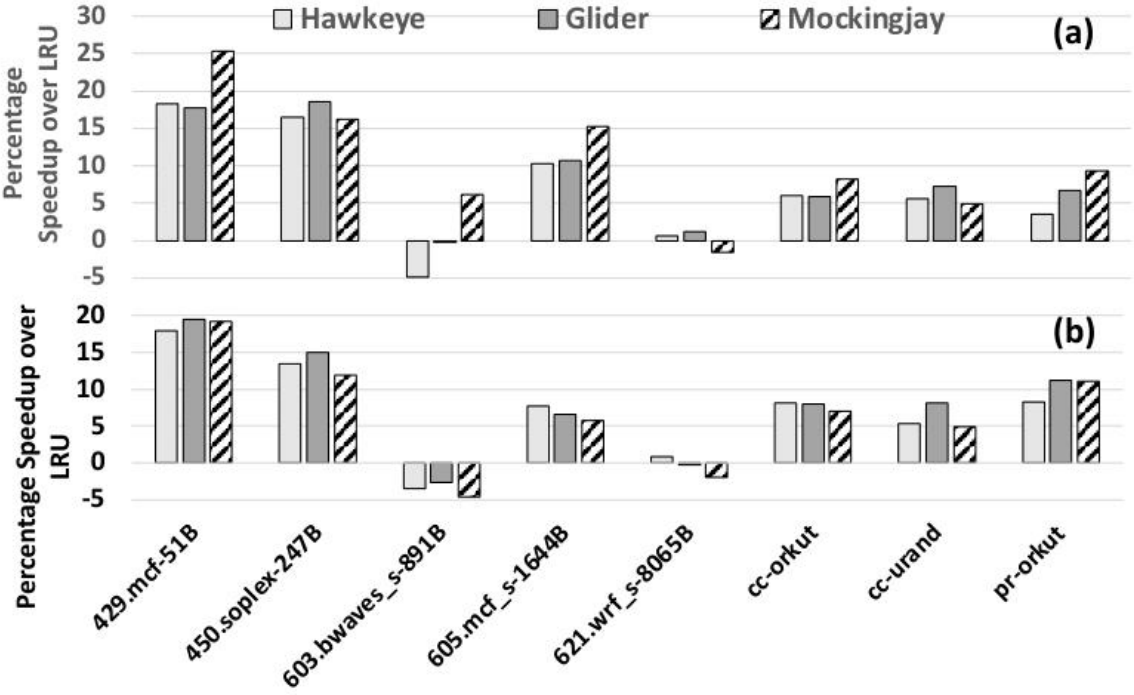



图 1 对比了 Hawkeye、Glider、Mockingjay 在两种预取配置下的加速比

## 2.2 核心技术架构：强化学习驱动的整体化设计

CHROME 的突破在于将 “替换 - 旁路 - 预取” 三者深度融合，并通过在线强化学习（RL） 实现动态适配，其架构可拆解为 “RL 建模逻辑”“多特征状态感知”“并发感知奖励机制” 三部分：

### 2.2.1 缓存管理的强化学习建模

CHROME 将整个缓存管理过程抽象为强化学习的 “智能体 - 环境” 交互模型，核心要素定义如下：

环境（Environment）：处理器与内存系统，包括多核心的并发内存访问、缓存的当前状态（如缓存块的占用情况、预取器的预测结果）。

智能体（Agent）：CHROME 框架本身，负责实时做出缓存决策。

状态(State)：描述当前缓存访问的关键特征，CHROME 选择 “程序计数器(PC) 签名 + 物理页号（PN)” 构成二维状态向量。

PC 签名：将 PC 与 “命中 / 未命中” 信息哈希结合，既能区分同一 PC 发起的不同访问类型，又能捕捉程序控制流特征（不同 PC 对应不同指令的内存访问行为）；

物理页号：反映数据访问特征，同一页内的数据访问模式往往相似，可补充 PC 无法覆盖的 “数据局部性” 信息。

此外，为区分 “需求访问” 与 “预取访问”，CHROME 会在 PC 签名中加入 “预取标记位”；在多核心场景下，还会加入 “核心 ID”，避免不同核心的访问行

为混淆。

动作（Action）：CHROME 针对不同缓存场景的决策，分为两类：

缓存未命中时：决定 “将新块写入缓存（并分配 3 种驱逐优先级之一）” 或 “直接旁路缓存”；

缓存命中时：更新当前块的驱逐优先级（3 种优先级对应 “优先保留”“普通” “优先淘汰”）。

目标：通过持续交互，学习最优策略以最大化 “累积奖励”，最终降低缓存缺失率、提升系统吞吐量。

### 2.2.2 多特征状态感知：精准捕捉内存访问行为

传统 RL-based 缓存方案常仅依赖单一特征（如仅用 PC），导致对访问模式的刻画不全面。CHROME 则整合 “控制流特征”（PC 相关）与 “数据访问特征”（物理页号、内存地址等），通过以下设计提升状态描述的准确性：

特征筛选：并非越多特征越好 —— 过多特征会增加存储与计算开销。CHROME 通过 “特征选择算法”，从候选特征（如最近 4 个 PC 序列、内存地址偏移量）中筛选出 “PC+PN” 组合，在 “刻画精度” 与 “硬件开销” 间取得平衡。

哈希优化：为避免状态向量维度爆炸（如 PC 为 64 位时，直接存储会导致 Q 表过大），CHROME 对 PC 签名、核心 ID、预取标记位进行哈希处理，将高维特征压缩为低维索引，同时保证不同特征的区分度。

具体可以参考图 2:

| Control-flow | Data-access | Combination |
|---|---|---|
| PC | Memory address | PC + delta |
| Sequence of last 4 PCs | Memory address delta | PC + page number |
| | Sequence of last 4 deltas | PC + page offset |
| | Page number | |
| | Page offset | |

图 2 列出了 CHROME 考虑的控制流、数据访问及组合特征

### 2.2.3 并发感知的奖励机制：结合局部性与系统级反馈

奖励函数是 RL 智能体的 “指挥棒”，直接决定策略优化方向。CHROME 的奖励机制突破传统 “仅看缓存命中 / 未命中” 的单一维度，结合数据局部性（缓存块是否被重用）与系统并发度（多核心访问的重叠情况），核心是引入 “并发平均内存访问时间（C-AMAT）” 模型：

C-AMAT 的作用：量化 “缓存访问效率” 与 “并发访问重叠度” 的综合影响。例如，若某核心的 LLC 访问时间（C-AMAT）超过主存访问延迟，说明该核心在 LLC 中获取数据的效率低于直接访问主存，存在 “LLC 阻塞”（缓存未带来性能收益，反而因排队延迟拖慢速度）。

四类核心奖励设计：CHROME 根据 “访问类型（需求 / 预取）”“是否重用” “是否 LLC 阻塞”，定义 4 种奖励（R_AC、R_IN、R_AC-NR、R_IN-NR），举例如下：

若需求访问命中缓存（R_AC^D）：给予 + 20 的正奖励，鼓励保留需求访问的

有用块；

若预取访问未被重用且核心处于 LLC 阻塞（R_IN-NR^OB）：给予 - 22 的负奖励，惩罚 “盲目预取 + 写入缓存” 的行为；

若旁路了未被重用的块且核心无 LLC 阻塞（R_AC-NR^NOB）：给予 + 10 的正奖励，鼓励有效旁路以避免缓存污染。

这种奖励机制确保 CHROME 的决策不仅关注单个缓存块的 “局部收益”，更考虑多核心并发下的 “系统级收益”，避免单一核心的优化损害整体性能。

## 2.3 硬件实现细节：低开销与高实用性

CHROME 需在硬件中实时运行，因此设计时严格控制存储与延迟开销，核心硬件组件包括 “Q 表（Q-Table）”“评估队列（EQ）”，且采用 “采样训练” 降低计算压力：

### 2.3.1 Q 表：高效存储状态 - 动作的 Q 值

Q 表的作用是存储 “某一状态下选择某一动作的预期长期奖励（Q 值）”，是 RL 智能体的 “经验库”。传统 Q 表若采用 “状态 × 动作” 的二维数组，会因状态数量庞大导致存储爆炸。CHROME 通过两点优化解决这一问题：

分块存储：将状态向量的两个特征（PC 签名、PN）拆分为独立的子 Q 表，查询时并行读取 “PC - 动作” 与 “PN - 动作” 的 Q 值，取最大值作为最终 Q 值。例如，查询状态（PC1, PN1）对应动作 “写入缓存” 的 Q 值时，分别查 “PC1 - 写入” 和 “PN1 - 写入” 的 Q 值，取较大者，既保证决策精度，又减少单表的存储量。

子表哈希划分：每个特征（如 PC 签名）对应的 Q 表再拆分为 4 个小的子表，通过哈希函数将特征值映射到对应子表的索引，读取时并行累加各子表的部分 Q 值，得到最终结果。这种设计将 Q 表的存储开销控制在 32KB（2 个特征 ×4 个子表 ×2048 条目 ×16 位 / 条目），远低于 Hawkeye 的 146KB、Glider 的 254KB。

图 3 展示了 Q 表查找的五阶段流水线结构（特征提取、索引生成、部分 Q 值读取、汇总计算、取最大值），能直观呈现 “分块存储 + 子表哈希划分” 的硬件实现流程，解释 Q 表如何在低延迟下完成高效查询:

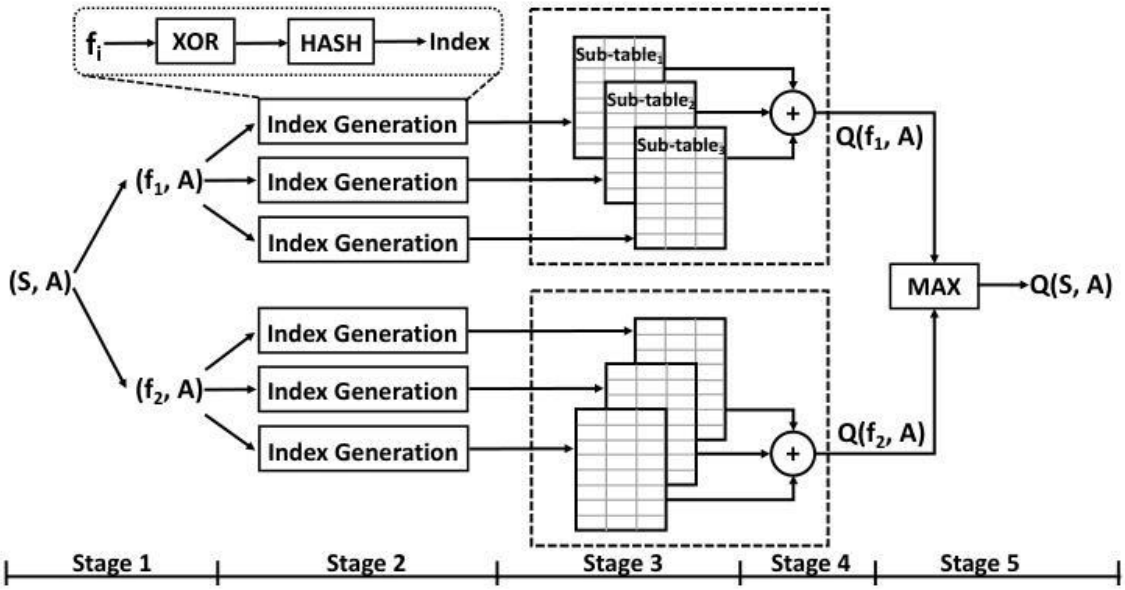



图 3 展示了 Q 表查找的五阶段流水线结构

### 2.3.2 评估队列（EQ）：延迟奖励的 “记账本”

缓存决策的效果无法立即判断（如预取的块是否有用，需等待后续是否被访问），因此 CHROME 设计 EQ 作为 “延迟奖励评估工具”，核心功能如下：

条目内容：每个 EQ 条目记录 “状态向量（S）、执行的动作（A）、访问类型（命中 / 未命中）、内存地址、奖励值（初始为空）”，采用 FIFO 队列结构，固定容量。

奖励分配逻辑：

当新的缓存访问请求到来时，若地址与 EQ 中某条目匹配（说明该地址之前的决策已产生结果），则根据 “当前访问是命中还是未命中” 分配奖励（如命中则给 R_AC，未命中则给 R_IN）；

若 EQ 条目被淘汰时仍未匹配到访问（说明该地址未被重用），则根据 “是否 LLC 阻塞” 分配 R_AC-NR 或 R_IN-NR；

分配奖励后，用 SARSA 算法更新 Q 表：$Q(S,A) = Q(S,A) + \alpha \times [R + \gamma \times Q(S',A') - Q(S,A)]$，其中 $\alpha$ 是学习率（控制更新幅度），$\gamma$ 是折扣因子（平衡即时与长期奖励）。

### 2.3.3 采样训练：降低计算复杂度

若对所有缓存集的访问都进行 RL 训练，会带来巨大的计算开销。CHROME 借鉴 “缓存集采样” 思想，仅选择 64 个缓存集（称为 “采样集”）进行完整的 RL 交互与 Q 表更新，其他缓存集直接复用采样集学到的策略。这一设计的依据是：内存访问模式在不同缓存集间具有一致性（同一程序的内存地址映射到不同缓存集，但访问规律相似）。通过采样训练，CHROME 在保证策略精度的同时，将计算开销降低到可接受范围 —— 硬件仅需处理 64 个集的状态查询与 Q 表更新，无需遍历所有缓存集。

## 2.4 性能优势：低开销下的全面超越

CHROME 在 16 核系统（采用 Intel Alder Lake 架构）、多种工作负载（SPEC CPU2006/2017、GAP）下的测试结果显示，其性能与实用性全面优于主流方案：

### 2.4.1 性能提升显著且稳定

对比传统基线：相较于经典的 LRU 算法，CHROME 在多核心同构工作负载（如 16 核同时运行相同的 GAP 图算法）下，性能平均提升 13.7%；在异构工作负载（16 核运行不同的 SPEC/GAP 程序）下，提升达 14.4%。

超越 SOTA 方案：在 16 核系统中，CHROME 比 Hawkeye（基于 Belady 最优策略仿真）快 6.6%，比 Glider（基于离线 LSTM 训练）快 6.9%，比 CARE（考虑并发的局部性优化）快 2.6%。核心原因是 CHROME 的整体化设计避免了策略孤立的缺陷，且 RL 动态适配能应对不同工作负载的差异。

图 4 对比了 CHROME 与 Hawkeye、Glider 等 SOTA 方案在 16 核系统同构工作负载下的加速比

图 5 展示了 4 核系统 SPEC 同构工作负载下各方案的加速比

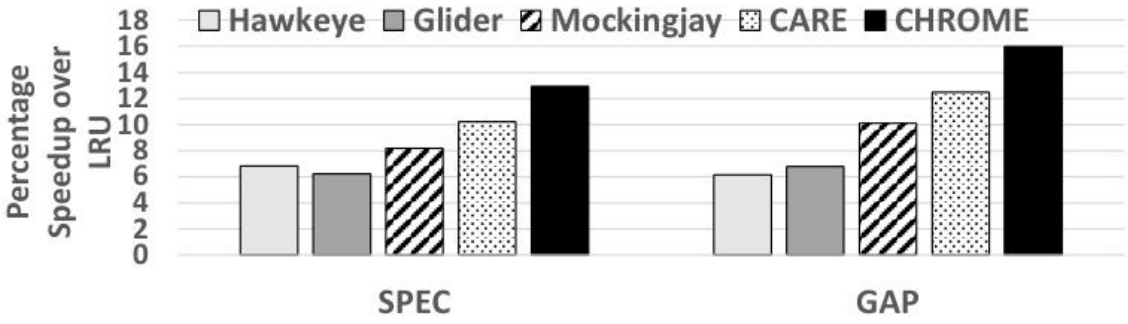



图 4 不同方案 16 核系统加速比对比图

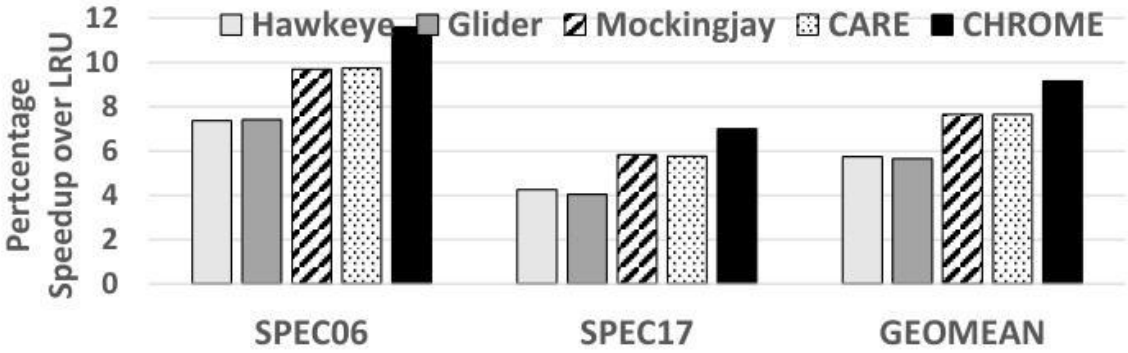



图 5 4 核系统 SPEC 同构工作负载下各方案的加速比

### 2.4.2 硬件开销极低

CHROME 的总存储开销仅 92.7KB，由三部分构成：Q 表（32KB）、EQ（12.7KB，64 个队列 ×28 条目 ×58 位 / 条目）、元数据（EPV 标记，48KB，每个缓存块

用 2 位记录驱逐优先级)。这一开销仅占 12MB LLC（4 核系统）的 0.75%，且不随 LLC 容量增加而增长（采样集数量固定为 64 个），远低于其他方案（如 Mockingjay 的 170.6KB、CARE 的 130.5KB），完全满足硬件实现的实用性要求。

### 2.4.3 扩展性优异

随着核心数增加（从 4 核到 16 核），CHROME 的性能优势进一步扩大：在 16 核同构工作负载下，CHROME 比 CARE 快 2.5%，而在 4 核系统中仅快 1.4%。原因是多核心并发导致缓存竞争更激烈，传统方案的静态策略难以协调，而 CHROME 的并发感知奖励机制（基于 C-AMAT）能实时优化缓存资源分配，减少核心间的干扰。

图 6 对比了 CHROME 与 N-CHROME（无并发感知反馈）在不同核心数下的性能，能进一步说明 “并发感知机制” 是 CHROME 扩展性优异的关键原因

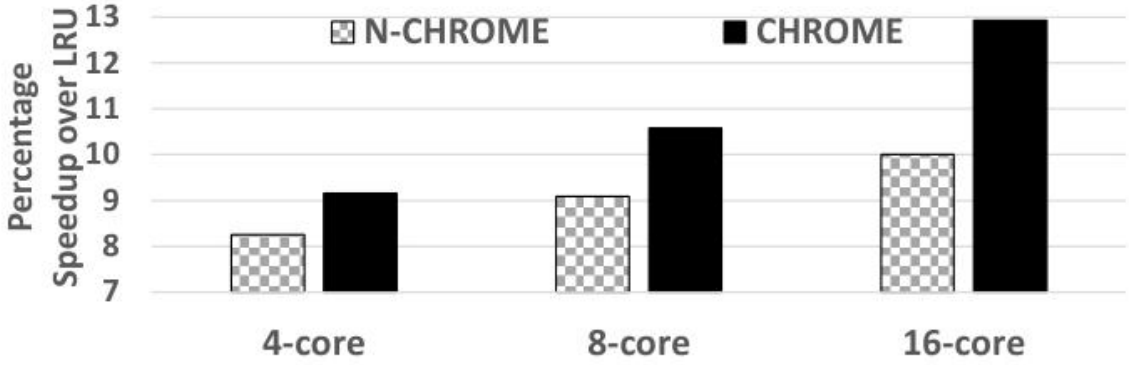



图 6 对比了 CHROME 与 N-CHROME 在不同核心数下的性能

## 3.缓存一致性协议：虚拟网络的死锁防护机制

### 3.1. 研究背景：虚拟网络死锁问题与传统方案的缺陷

在多核处理器的缓存一致性协议中，虚拟网络（VN）是避免消息循环依赖引发死锁的核心技术，但传统基于 “消息依赖链长度” 的 VN 数量计算方法存在根本性缺陷 —— 既不充分也不必要，无法适配多目录、任意互连网络拓扑的通用系统模型。

### 3.1.1 传统方案的 “不充分性”：静态依赖链分析导致死锁

传统教材中，VN 数量由 “最长消息依赖链” 决定（如请求→转发请求→响应的链长为 3，故需 3 个 VN），但该方法在多目录系统中会失效。例如基于 Nagarajan 等人 [11] 的 MSI 协议（论文中 Figures 1、2 为该协议的缓存与目录控制器状态表),在“3 个缓存 + 2 个目录”的配置下,会因转发请求(FwdGetM)的阻塞形成循环依赖。

当缓存 C1、C2 分别向对方目录请求缓存块，且目录发送的 FwdGetM 被对方

缓存阻塞时，若 FwdGetM 均分配到同一 VN，会形成 “C1 的 FwdGetM 等待 C2 的 FwdGetM 释放通道” 的循环，最终引发死锁。如图 7:

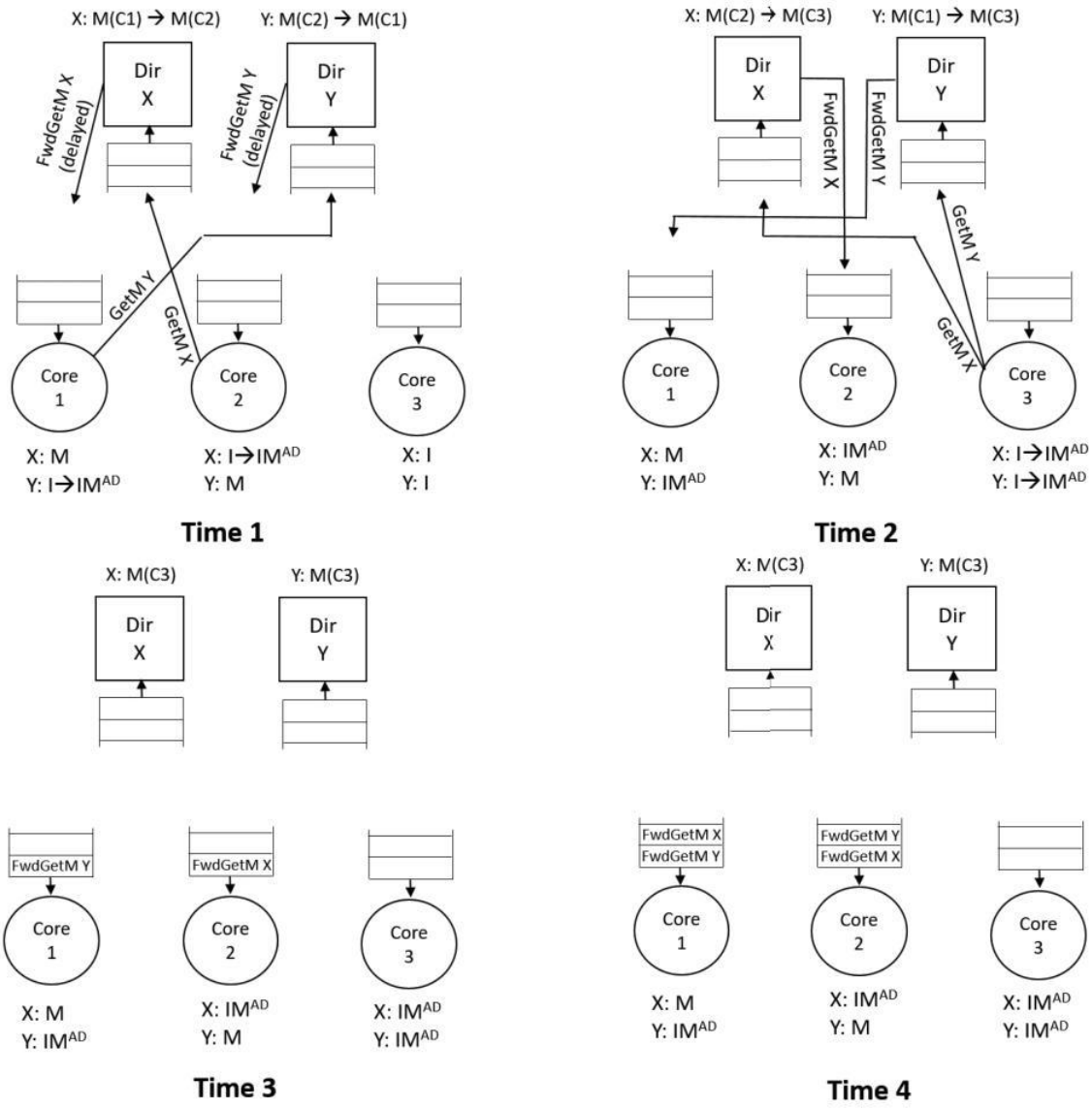



图 7 上述死锁场景的时序图

### 3.1.2 传统方案的 “不必要性”：过度配置浪费资源

部分协议无需按依赖链长度配置 VN。例如完全非阻塞目录协议（如 MOSI、MOESI）支持并发处理请求，无消息阻塞，仅需 1 个 VN 即可；而工业界的 CHI 协议规范要求 4 个 VN，但论文分析表明其仅需 2 个 VN 即可避免死锁。这种 “过度配置” 会增加 NoC 的硬件开销（如缓冲区、路由逻辑），导致功耗上升与延迟增加。

## 3.2. 核心技术：形式化建模与死锁分析框架

论文通过定义 “消息依赖关系” 与 “网络排队模型”，构建了普适性的死锁判定体系，为 VN 数量计算提供理论基础。

### 3.2.1 三大核心关系：刻画协议与网络的交互

论文定义三类静态关系，将缓存一致性协议的死锁风险转化为可量化的模型：

causes 关系（消息触发关系）：若消息 $m_1$在某事务中触发 $m_2$，则 $m_1$→causes→$m_2$（如 Read 请求触发 FwdRead 响应）；

stalls 关系（消息阻塞关系）：若消息 $m_0$使控制器进入 transient 状态并阻塞 $m_1$，则 $m_0$→stalls→$m_1$（如 MSI 协议中 GetS 阻塞后续 GetM）；

waits 关系（消息等待关系）：由 stalls 与 causes 推导，若 $m_1$被 $m_0$阻塞且需等待 $m_0$触发的后续消息完成，则 $m_1$→waits→$m_2$（如 GetM 等待 FwdGetM 的响应 Data）。

### 3.2.2 网络排队模型：queues 关系与死锁条件

论文假设互连网络（ICN）为 “任意拓扑”，定义 queues 关系：若消息 $m_1$与 $m_2$分配到同一 VN，且 $m_1$可能阻塞在 $m_2$之后，则 $m_1$→queues→$m_2$。此时，死锁的充分必要条件为 “(waits ∪ queues) 关系存在循环”—— 即消息间形成 “等待 - 排队” 的闭环。

## 3.3. 协议分类与最小 VN 数量判定

基于上述框架，论文将缓存一致性协议分为三类，并给出每类的最小 VN 需求，核心结论通过实验与模型验证（Murphi 模型检查）支撑。

### 3.3.1 三类协议的 VN 需求差异

Class 1（协议死锁）：存在原生 waits 循环，即使每个消息独立 VN 仍死锁（如设计缺陷的 MSI 变体），需通过协议修正解决；

Class 2（不可避免 VN 死锁）：waits 关系存在循环（如 FwdGetM→waits→FwdGetM），需为每个缓存块分配独立 VN（不切实际），应避免使用；

Class 3（实用协议）：无 waits 循环，仅需 2 个 VN 即可避免死锁（如 CHI、MOESI），是工业界主流协议类型。

### 3.3.2 CHI 协议的验证：2 个 VN 足够

论文首次对 CHI 协议进行形式化建模，发现其消息可分为 “请求类”（如 CleanUnique、ReadShared）与 “非请求类”（如 Inv、DataResp）两类。将请求类分配到 VN1，非请求类分配到 VN2，可打破所有 (waits ∪ queues) 循环。

图 8 展示 CleanUnique 请求触发 Inv、InvAck、Resp、Comp 的交互流程.

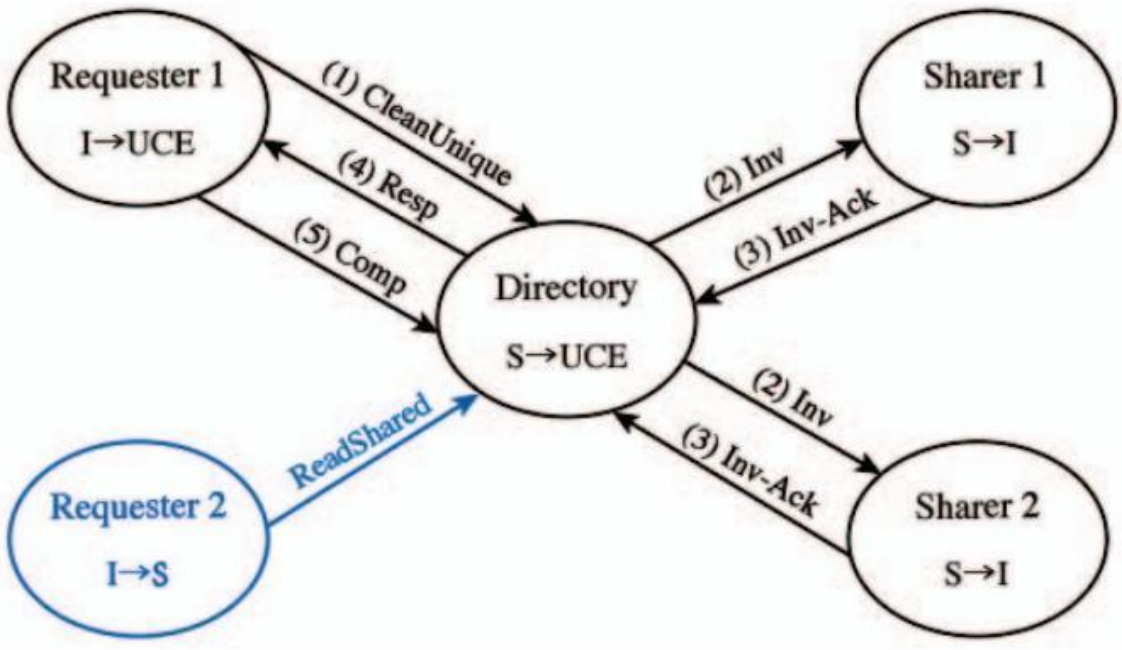



图 8 CHI 协议的事务时序图

## 3.4. 实验验证：Murphi 模型与结果

论文通过 Murphi 模型检查器验证结论，重点测试 “多目录 + 任意 ICN” 场景，核心实验结果如下：

### 3.4.1 模型配置与验证方法

系统模型：3 个缓存、2 个目录、2 个地址（模拟多目录竞争）；

ICN 建模：采用 “全局 FIFO + 端点缓冲” 模拟任意拓扑的消息排队（如 Section VII 的 ICN 模型图）；

验证指标：是否存在死锁路径、VN 数量与死锁概率的关系。

图 9 展示每个 VN 包含 “全局缓冲 + 端点缓冲”，支持模拟消息的任意排队与重排序，解释实验如何覆盖 “任意 ICN 拓扑” 的通用场景.

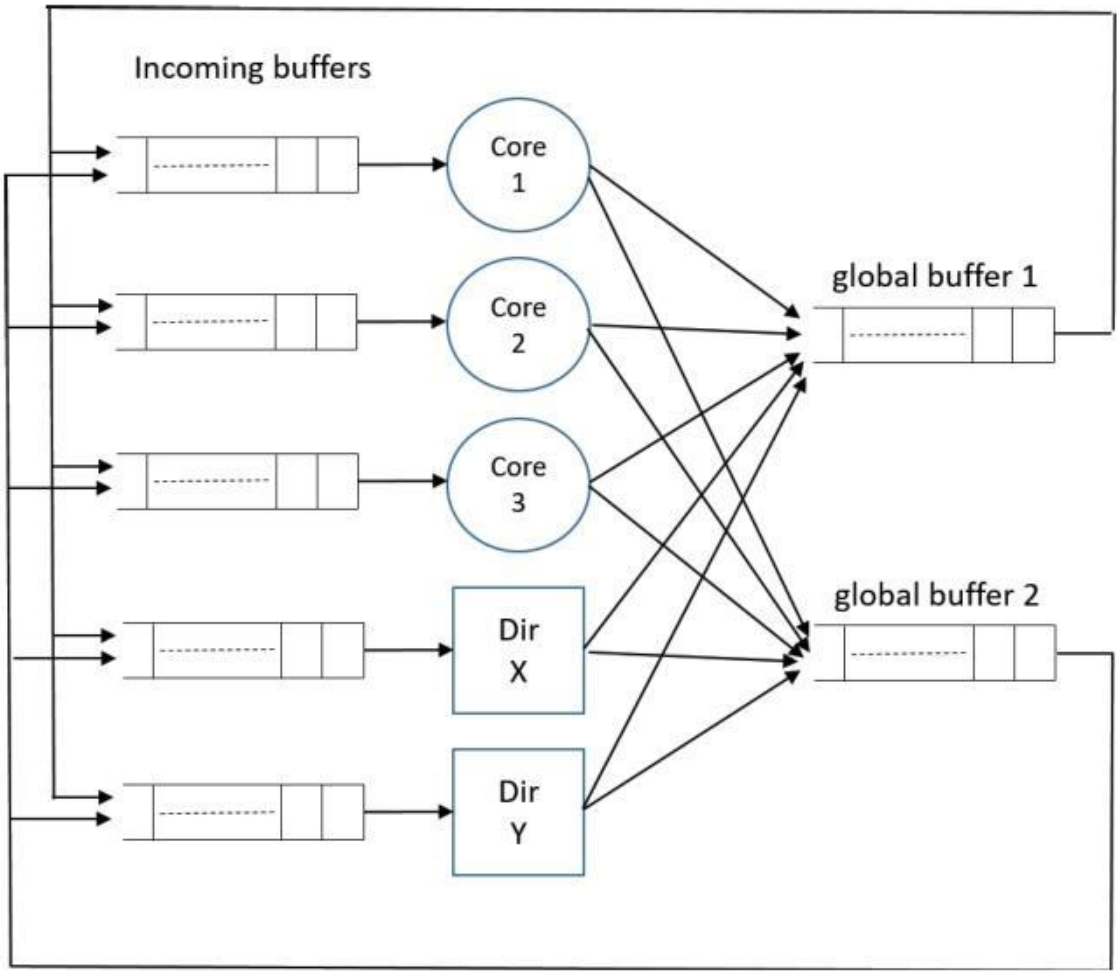



图 9 ICN 模型示意图

### 3.4.2 关键实验结果

Class 3 协议：CHI、MOESI 等协议在 2 个 VN 配置下，模型检查未发现死锁（即使模拟 1e6 个周期）；

传统方案对比：Hawkeye、Glider 等基于静态策略的协议，在多核心场景下需 3 个 VN，而 CHI 仅需 2 个 VN 即可达到同等可靠性；

开销对比：2 个 VN 比 CHI 规范的 4 个 VN 减少约 50% 的 NoC 存储开销（缓冲区面积）与 30% 的静态功耗。

# 4.安全高效缓存：Maya 的存储优化与攻击防护

Maya Cache 是印度理工学院孟买分校团队提出的创新末级缓存（LLC）设计，核心是在保障高安全性的同时优化存储效率，解决了传统安全缓存 “性能 - 安全 - 存储” 难以平衡的痛点。以下从研究背景、核心设计、安全机制、性能表现、硬件开销五大维度，结合论文原图展开详细解析。

## 4.1 研究背景：安全缓存的存储与性能矛盾

现代多核处理器的共享末级缓存（LLC）易受跨核冲突攻击（如 Prime+Probe、Flush+Reload），而全相联缓存 + 随机替换策略是抵御这类攻击的有效方案，但直接设计大规模全相联缓存不切实际。现有方案存在明显缺陷：

### 4.1.1. 传统随机化缓存的安全与性能局限

CEASER、Scatter-Cache 等随机化缓存通过地址映射随机化抵御攻击，但仍存在概率性冲突风险，无法提供绝对安全；SassCache 虽提升安全性，却导致超过 4% 的性能下降，难以满足实际需求。

### 4.1.2. Mirage 缓存的存储开销问题

Mirage 通过解耦标签存储与数据存储，提供全相联缓存的 “illusion”，其随机全局替换策略能有效防御冲突攻击，但需额外配置无效标签条目，导致存储开销增加 20%、静态功耗上升 18.16%。例如 16MB 基准缓存的存储需求为 16.91MB，而 Mirage 需 20.31MB，在 32 核系统中额外开销高达 13.62MB。

### 4.1.3. 关键观察：缓存数据的低重用特性

论文发现,LLC 中超过 80% 的数据块“一经安装即失效”(dead on arrival)，生命周期内从未被重用，却占用大量数据存储资源；且数据存储条目占用的存储量约为标签存储条目的 8 倍。这一洞察成为 Maya 设计的核心依据 —— 通过筛选重用数据进入数据存储，可在不牺牲性能的前提下缩减数据存储规模，抵消标签存储的额外开销。

图 10 对比了基准缓存与 Mirage 在 SPEC CPU2017 和 GAP 基准测试中失效块的占比,直观展示 “超过 80% 数据块无重用” 的核心观察，为 Maya “缩减数据存储” 的设计提供数据支撑

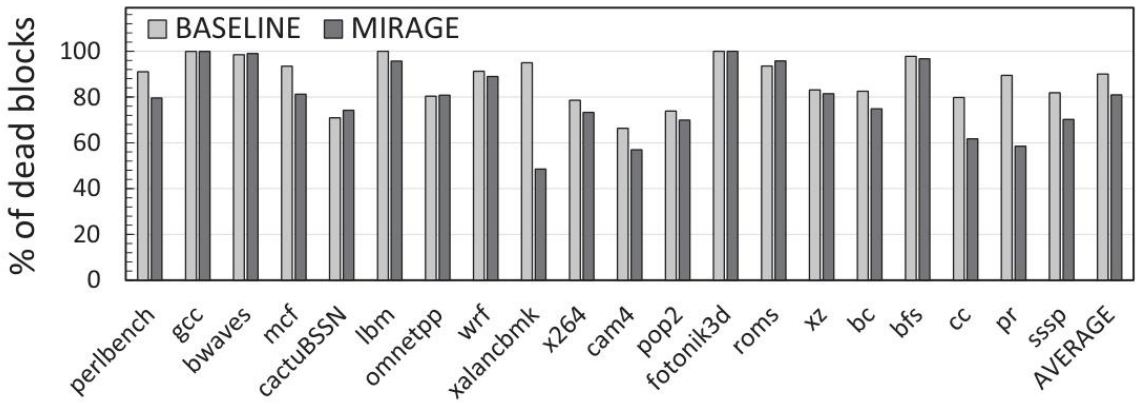



图 10 基准缓存与 Mirage 在 SPEC CPU2017 和 GAP 基准测试中失效块的占比

## 4.2 核心设计：解耦架构与重用驱动的存储优化

Maya 的核心创新是 “扩展标签存储保障安全与重用检测，缩减数据存储提升效率”，采用解耦的标签 - 数据存储架构，配合优先级管理与智能调度策略，实现安全、性能与存储的平衡。

### 4.2.1. 标签与数据存储设计

（1）斜列相联标签存储

标签存储分为两个独立斜列（skew），每个斜列采用独立哈希函数映射缓存集，确保每个缓存行可映射到两个不同集合，提升负载均衡能力。标签条目包含 70 位：40 位标签位、3 位一致性位（MOESI 协议）、1 位优先级位、18 位正向指针（FPTR）、8 位安全域 ID（SDID）。

优先级位：区分优先级 0（仅标签，无对应数据）和优先级 1（标签 + 数据）条目；

FPTR：指向数据存储中的对应条目（仅优先级 1 有效）；

SDID：隔离不同安全域的缓存填充，抵御共享内存攻击（如 Flush+Reload），支持最多 256 个域。

（2）精简数据存储

数据存储仅保留被重用的数据块，每个条目包含 512 位数据（64B 缓存行）和 19 位反向指针（RPTR），RPTR 指向对应的标签条目，总位数为 531 位。相较于 16MB 基准缓存的 16384KB 数据存储，Maya 仅需 12744KB，缩减 28.11%。

（3）额外标签通道

标签存储除基础通道外，新增两类通道：

无效标签通道：每斜列 6 个，确保缓存行安装时总有无效标签可用，避免集相联驱逐（SAE），保障安全性；

重用检测通道（reuse ways）：每斜列 3 个，专门存储优先级 0 条目，监测数据块的重用行为。

图 11 展示了 Maya 的整体架构，清晰呈现标签存储（含无效、优先级 0、优先级 1 条目）与数据存储的解耦关系，以及正向指针、反向指针的关联逻辑，辅助理解 “标签 - 数据” 的映射机制




图 11 Maya 的整体架构

### 4.2.2. 插入与驱逐策略

（1）负载感知插入策略

缓存行安装时，根据两个斜列对应集合的无效标签数量，选择无效标签更多的集合插入（负载感知斜列选择），避免单一集合过载导致 SAE。插入时默认分配优先级 0，仅存储标签，不占用数据存储资源。

（2）重用驱动的优先级升级

当优先级 0 条目被再次访问（检测到重用）时，升级为优先级 1，分配数据存储条目，将数据载入并建立 FPTR 与 RPTR 的关联；若数据存储已满，则通过全局随机数据驱逐策略，随机选择一个优先级 1 条目降级为优先级 0，释放数据存储空间。

（3）全局随机驱逐策略

标签驱逐：新增优先级 0 条目时，若重用通道已满，随机驱逐一个优先级 0 条目（全局随机标签驱逐），确保无效标签数量稳定；

数据驱逐：优先级 0 升级为优先级 1 时，若数据存储已满，随机驱逐一个优先级 1 条目（全局随机数据驱逐），避免攻击者通过驱逐顺序推断缓存状态。

3. 实现参数优化

Maya 的关键参数经过权衡设计：每斜列 6 个基础通道（对应优先级 1 条目）、3 个重用通道（优先级 0）、6 个无效通道，标签存储共 480K 条目（4.1MB），数据存储 192K 条目（12.44MB）。采用 12 轮 PRINCE 密码实现地址随机化，仅增加 3 个周期延迟，加上标签 - 数据间接查找的 1 个周期，总额外延迟为 4 个周期，影响可控。

## 4.3 安全机制：抵御冲突攻击与共享内存攻击

Maya 通过多重机制构建强安全保障，核心目标是使攻击者无法通过冲突或共享内存通道获取敏感信息，安全级别达到 “系统生命周期内无法发起有效攻击”。

### 4.3.1. 冲突攻击防御：避免集相联驱逐（SAE）

SAE 是冲突攻击的前提（攻击者需通过 SAE 构建驱逐集），Maya 通过以下设计避免 SAE：

充足无效标签：每斜列 6 个无效通道，配合负载感知插入，确保缓存行安装时总有无效标签可用；

全局随机驱逐：标签与数据驱逐均为全局随机，攻击者无法预测驱逐顺序，无法构建稳定的驱逐集。

论文通过 “桶球模型” 模拟验证，在 15 路 / 斜列的配置下，SAE 发生概率极低，需 $4\times10^{32}$ 次缓存行安装（约 $10^{16}$年）才能出现一次，远超系统生命周期。

### 4.3.2. 共享内存攻击防御：安全域隔离（SDID）

对于 Flush+Reload、Flush+Flush 等共享内存攻击，Maya 通过 8 位 SDID 为每个标签条目标记安全域，不同域的缓存行即使地址相同也会被视为独立条目，实现物理隔离。这确保攻击者无法通过共享缓存行的访问延迟推断受害者的行为，从根源上阻断共享内存攻击路径。

### 4.3.3. 其他攻击防御补充

时序攻击：不同安全域的条目隔离，攻击者无法通过优先级 0/1 条目的访问延迟差异获取信息；

占用攻击：Maya 虽不直接防御占用攻击，但实验表明其对 AES 和模幂运算的占用攻击难度与全相联缓存接近（归一化值 0.996 和 0.992），远高于 16 路组相联缓存（0.85 和 0.63），未增加攻击风险。

图 12 对比了 16 路组相联缓存、Maya 与全相联缓存在占用攻击下的破解所需加密次数，验证 Maya 不会加剧占用攻击风险

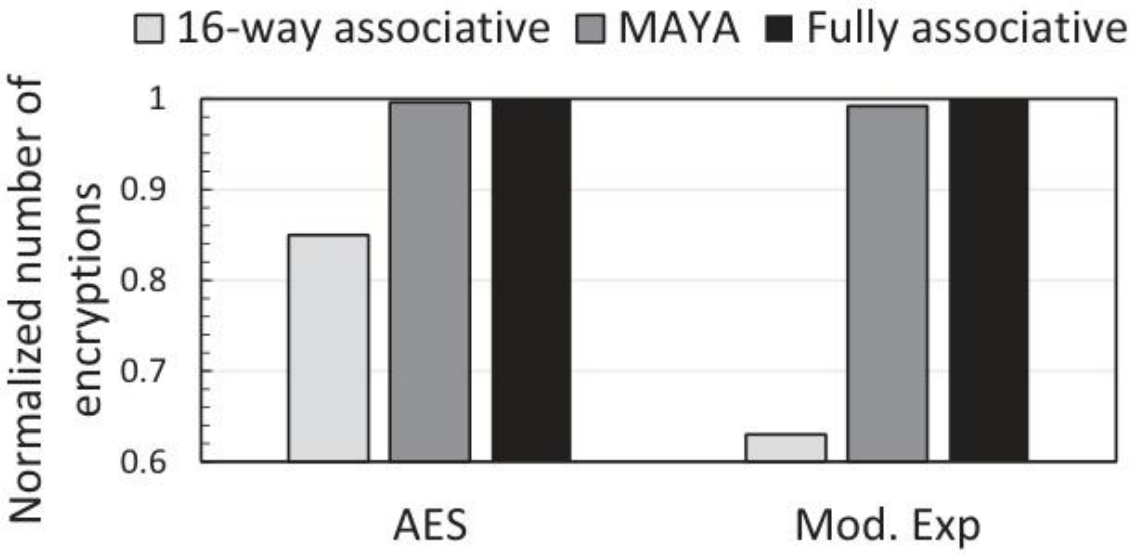



图 12 加密次数对比图

## 4.4 性能表现：多工作负载下的稳定性与优势

论文基于 ChampSim 模拟器，在 8 核系统上通过 SPEC CPU2017、GAP 基准测试及异构混合工作负载，验证 Maya 的性能表现，基准为 16MB 16 路组相联缓存（SRRIP 替换）。

### 4.4.1. 同构工作负载性能

SPEC CPU2017：Maya 平均性能比基准提升 0.20%，部分高冲突工作负载（如 mcf、wrf）性能提升显著（超过 70% 的核心间干扰减少），抵消了额外延迟的影响；低失效块占比的工作负载（如 cam4、cactuBSSN）因数据存储缩减略有性能下降，但幅度可控；

GAP：平均性能比基准提升 5%，其中 PageRank（pr）工作负载提升达 50%，因基准预取策略效果差，Maya 的重用筛选机制更适配图算法的访问模式。

### 4.4.2. 异构工作负载性能

21 组异构混合工作负载中，Maya 平均性能比基准提升 1.5%：低 MPKI（缓存缺失率）混合负载提升超过 4%（核心间干扰减少）；中高 MPKI 负载因工作集较大，性能略有下降，但幅度小于 1%，展现出良好的适应性。【此处可插入论文 Fig. 10】（说明：Fig. 10 对比了 Maya 与 Mirage 在低、中、高 MPKI 异构混合负载下的性能，体现 Maya 在不同冲突程度负载下的稳定性，放置于 “异构工作负载性能” 小节，补充异构场景的性能数据。）

### 4.4.3. 扩展性与敏感性分析

核心数扩展：16 核和 32 核系统中，Maya 仍保持性能提升，且性能损失随核心数增加趋于饱和，适配多核扩展；

LLC 容量敏感性：6MB 数据存储的 Maya 性能最优，容量超过 24MB 后，性能提升边际递减，因工作集已大部分命中 LLC；

重用通道敏感性：3 个重用通道为最优配置，1 个通道时性能略有下降（部分重用数据未被捕捉），5 个及以上通道时标签查找延迟增加，导致性能下滑。

## 4.5 硬件开销：存储、面积与功耗优化

Maya 通过数据存储缩减实现显著的硬件开销优化，同时保持标签存储的安全扩展，整体开销优于基准缓存和 Mirage。

1. 存储开销

Maya 总存储为 16994KB，比基准缓存（17312KB）节省 2%；而 Mirage 总存储为 20856KB，开销增加 20%。存储节省源于数据存储的大幅缩减（12744KB vs 基准 16384KB），抵消了标签存储的扩展（4200KB vs 基准 928KB）。

### 4.5.2. 面积与功耗

基于 7nm FinFET 技术的 P-CACTI 仿真结果显示：

面积：Maya 为 10.686mm²，比基准（14.868mm²）节省 28.11%；Mirage 为 15.887mm²，开销增加 6.86%；

静态功耗：Maya 为 588mW，比基准（622mW）降低 5.46%；Mirage 为 735mW，开销增加 18.16%；

动态能耗：Maya 的读 / 写能耗分别为 2.661nJ / 次和 4.116nJ / 次，比基准降低 15.55% 和 11.40%，因数据存储更小，访问能耗更低。

### 4.5.3. 与安全分区技术的对比

相较于 DAWG、Page Coloring、BCE 等安全分区技术，Maya 无需牺牲大量性能：DAWG 性能下降 16%，Page Coloring 下降 19%，BCE 下降 9%，而 Maya 仅实现 0.20% 的性能提升，同时存储开销更低（BCE 增加 2% 存储，Maya 节省 2%）。

## 5.研究共性与未来展望

三篇文档围绕缓存系统的核心挑战展开，呈现出显著的共性特征：均针对传统设计的孤立性缺陷，采用整体化思维提出创新方案；注重动态适应性，通过机器学习、形式化建模等先进技术提升系统对复杂工作负载的适配能力；兼顾实用性与高效性，在保障性能的同时控制硬件开销。

这些研究为缓存系统的进一步优化指明了方向：未来可探索强化学习与形式化方法的融合应用，实现性能优化与死锁防护的协同设计；在安全防护方面，可结合缓存分区、加密等技术，构建多层次安全体系；同时需关注多核心、大数据场景下的缓存扩展问题，提升系统的可扩展性。

## 总结

上完李老师的课,我对计算机体系结构有了更深的了解,对于涉及到系统底层逻辑有了了解,让我知道了流水线技术与并行处理的重要性, 通过重叠执行指令来提高指令吞吐量。但流水线设计并非完美，数据相关、控制相关、结构相关等问题都会影响流水线效率, 课程中介绍的定向技术、分支预测等解决方案让我了解到 cpu 设计的精妙之处.感谢老师的悉心教导,这段学习经历将成为我专业成长道路上的重要基石，激励我不断探索、持续进步。

## 参考文献

[1] Lu X Y, Najafi H, Liu J, Sun X H. CHROME: Concurrency-Aware Holistic Cache Management Framework with Online Reinforcement Learning(C)//2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA). Piscataway: IEEE, 2024: 1-12.

[2] Li W H, Goens A, Oswald N, Nagarajan V, Sorin D J. Determining

the Minimum Number of Virtual Networks for Different Coherence Protocols(C)//2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). New York: ACM Press; Piscataway: IEEE, 2024: 182-197.

[3] Bhatla A, Navneet, Panda B. The Maya Cache: A Storage-efficient and Secure Fully-associative Last-level Cache(C)//2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). New York: ACM Press; Piscataway: IEEE, 2024: 32-44.

| | | 评价内容 | 权重 | 得分 |
|---|---|---|---|---|
| 课程报告 | | 是否准确提炼每篇论文的研究问题、核心方法、关键结论与局限性；是否能对比分析不同论文的研究视角、方法差异或结论互补性；是否避免简单罗列单篇论文内容，而是围绕主题形成有逻辑的整合性阐述。 | 0.3 | |
| | | 是否能指出所选论文在研究设计、数据支撑、结论推导等方面的优点或不足；是否结合主题背景，分析论文研究的学术价值、实践意义或待解决的问题；是否能提出自己对主题的初步观点，且观点能以论文内容为依据支撑。 | 0.3 | |
| | | 结构是否清晰（如引言、论文综述、分析讨论、总结等模块完整且逻辑连贯）；是否规范引用论文（如参考文献格式统一、标注位置准确，无抄袭或不当引用）；语言表达是否符合学术写作要求（简洁、准确、专业，无口语化或逻辑混乱的表述）。 | 0.4 | |
| 合计 | | | | |
| 指导教师（签章）：李建华　　2025 年 12 月　日 | | | | |